

Rapport Projet fin d'année

5ème année Ingénierie Informatique et Réseaux

Sous le thème

Systeme de Détection d'Anomalies Comportementales Personnelles (PBADS)

Année universitaire 2025/2026

Réalisé par :

EL ALAMI Mohamed Majd & LANDRICHI Leila

Encadré par :

Pr. Benyoussef Marwa

Année universitaire 2025/2026

DÉDICACE

Nous dédions ce travail à toutes les personnes qui nous ont soutenues tout au long de ce projet.

À Pr. Benyoussef Marwa, notre encadrante pédagogique, pour son accompagnement attentif, ses conseils pertinents et son soutien constant tout au long de cette expérience enrichissante.

À l'École Marocaine des Sciences de l'Ingénieur (EMSI), pour la qualité de la formation dispensée et les opportunités offertes aux étudiants dans le cadre de leur développement professionnel.

À nos familles et à notre entourage, pour leur présence, leurs encouragements et leur soutien moral, qui nous ont permis de mener à bien ce projet.

REMERCIEMENTS

Nous tenons à exprimer notre profonde gratitude à toutes les personnes qui ont contribué à la réalisation de ce projet de fin d'année.

Nous adressons nos remerciements les plus sincères à Pr. Benyoussef Marwa, notre encadrante pédagogique à l'École Marocaine des Sciences de l'Ingénieur (EMSI), pour son expertise, sa disponibilité, ses précieux conseils et son accompagnement académique tout au long de ce projet. Sa guidance, sa bienveillance et ses orientations constructives ont été essentielles pour la réussite de ce travail.

Nous remercions également tous nos professeurs et encadrants de l'EMSI pour la qualité de leur enseignement et leur accompagnement tout au long de notre formation.

Enfin, merci à toutes celles et ceux qui, de près ou de loin, ont contribué à la réalisation de ce travail.

À tous... merci.

ACKNOWLEDGMENTS

We would like to express our deep gratitude to everyone who contributed to the completion of this end-of-year project.

Our sincere thanks go to Pr. Benyoussef Marwa, our academic supervisor at the Moroccan School of Engineering Sciences (EMSI), for her expertise, availability, and valuable guidance. Her support and constructive feedback were essential throughout this experience.

We would also like to thank all our professors and academic mentors at EMSI for the quality of education and the support they have provided throughout our studies.

Finally, thank you to everyone who, directly or indirectly, contributed to the development of this project.

To all... thank you.

Résumé

Ce travail présente la conception et la réalisation d'un **Système de Détection d'Anomalies Comportementales Personnelles (PBADS)** basé sur une architecture microservices. Le système analyse les données quotidiennes des utilisateurs (sommeil, activité physique, alimentation, humeur) et détecte en temps réel les anomalies comportementales à l'aide d'algorithmes d'apprentissage automatique non supervisés.

La solution repose sur une **architecture microservices** avec 13 services distincts, incluant un service d'authentification (JWT), un service de données, un service de modèles ML, un service d'inférence pour la détection d'anomalies, un service d'alertes, et un service de recommandations alimenté par Google Gemini AI. La communication entre services utilise Kafka pour la messagerie asynchrone et HTTP/REST pour les appels synchrones, avec un mécanisme de fallback pour garantir la résilience.

Le modèle de données structure les entités principales : **User**, **DailyData**, **Alert** et **Recommendation**. Le système utilise des algorithmes de détection d'anomalies (Isolation Forest et One-Class SVM) pour identifier les écarts par rapport aux comportements normaux, génère des alertes avec des niveaux de sévérité (NORMAL, LOW, MEDIUM, HIGH), et fournit des recommandations personnalisées via l'intégration avec Google Gemini AI.

Les bénéfices observés : **détection automatique des anomalies**, **recommandations personnalisées**, **notifications en temps réel**, et **visualisation interactive** des données comportementales via un tableau de bord avec filtres temporels. Le système offre une scalabilité élevée grâce à l'architecture microservices et une tolérance aux pannes via Resilience4j.

Mots-clés : Détection d'anomalies, Machine Learning, Microservices, Spring Boot, Kafka, PostgreSQL, Isolation Forest, One-Class SVM, Google Gemini AI, Recommandations personnalisées, Architecture microservices.

Abstract

This work presents the design and implementation of a **Personal Behavior Anomaly Detection System (PBADS)** based on a microservices architecture. The system analyzes users' daily data (sleep, physical activity, diet, mood) and detects behavioral anomalies in real-time using unsupervised machine learning algorithms.

The solution is built on a **microservices architecture** with 13 distinct services, including an authentication service (JWT), a data service, an ML model service, an inference service for anomaly detection, an alert service, and a recommendation service powered by Google Gemini AI. Inter-service communication uses Kafka for asynchronous messaging and HTTP/REST for synchronous calls, with a fallback mechanism to ensure resilience.

The data model structures the main entities : *User*, *DailyData*, *Alert* and *Recommendation*. The system uses anomaly detection algorithms (Isolation Forest and One-Class SVM) to identify deviations from normal behaviors, generates alerts with severity levels (NORMAL, LOW, MEDIUM, HIGH), and provides personalized recommendations through integration with Google Gemini AI.

The observed benefits include : **automatic anomaly detection**, **personalized recommendations**, **real-time notifications**, and **interactive visualization** of behavioral data through a dashboard with temporal filters. The system offers high scalability through the microservices architecture and fault tolerance via Resilience4j.

Keywords : Anomaly Detection, Machine Learning, Microservices, Spring Boot, Kafka, PostgreSQL, Isolation Forest, One-Class SVM, Google Gemini AI, Personalized Recommendations, Microservices Architecture.

Table des matières

Dédicace	1
Remerciements	2
Acknowledgments	3
Résumé	4
Abstract	5
Table des matières	7
Liste des Figures	8
Liste des Tableaux	9
Liste des abréviations	11
Introduction générale	12
Chapitre 1. Contexte général et planification du projet	15
1.1 Introduction	16
1.2 Contexte général	16
1.2.1 Environnement et motivation	16
1.2.2 Situation initiale et problématique	16
1.2.3 Objectif de la solution	17
1.2.4 Architecture microservices	18
1.3 Conclusion	18
Chapitre 2. Modélisation	19
2.1 Modélisation du système	20
2.1.1 Contexte général	20
2.1.2 Enjeux et objectifs	21
2.1.3 Méthodologie adoptée	21
2.2 Étude Conceptuelle	22
2.2.1 Méthodologie	22

2.2.2	Les acteurs principaux	23
2.2.3	Le diagramme de cas d'utilisation	23
2.2.4	Diagramme de classes	25
2.2.5	Les diagrammes de séquence	27
2.3	Conclusion	30
Chapitre 3. Interfaces et Réalisation de l'application		31
3.1	Technologies et Architecture	32
3.1.1	Architecture microservices	32
3.1.2	Technologies Frontend	32
3.1.3	Technologies Backend	33
3.1.4	Machine Learning	34
3.2	Interfaces Utilisateur	35
3.2.1	Vue Utilisateur	35
3.2.2	Vue Administrateur	42
3.3	Conclusion	43
Conclusion générale		44
Bibliographie		47

Table des figures

2.1	Diagramme de cas d'utilisation du système PBADS	24
2.2	Diagramme de classes du système PBADS	25
2.3	Diagramme de séquence - Administrateur interagissant avec la plateforme .	28
2.4	Diagramme de séquence - Modèle ML interagissant avec la base de données et la plateforme	29
3.1	Page d'inscription - Création de compte utilisateur	35
3.2	Page de connexion - Authentification utilisateur	36
3.3	Tableau de bord - Visualisation des tendances comportementales	37
3.4	Tableau de bord - Alertes et recommandations IA	38
3.5	Formulaire de saisie des données quotidiennes	39
3.6	Notification push - Alerte d'anomalie détectée	40
3.7	Page de consultation des alertes	41
3.8	Interface de gestion des utilisateurs (Administrateur)	42

Liste des tableaux

2.1	Acteurs principaux du système PBADS	23
-----	---	----

Liste des abréviations

Abréviation	Description
API	Application Programming Interface (Interface de Programmation d'Application).
PBADS	Personal Behavior Anomaly Detection System (Système de Détection d'Anomalies Comportementales Personnelles).
ML	Machine Learning (Apprentissage Automatique).
AI	Artificial Intelligence (Intelligence Artificielle).
JWT	JSON Web Token (Jeton d'authentification basé sur JSON).
CRUD	Create, Read, Update, Delete (Créer, Lire, Mettre à jour, Supprimer).
DB	Database (Base de Données).
EMSI	École Marocaine des Sciences de l'Ingénieur.
JPA	Java Persistence API (utilisé via Spring Data JPA pour l'accès et la gestion des données).
REST	Representational State Transfer, style architectural pour les APIs exposées par le backend.
PostgreSQL	Système de gestion de base de données relationnelle utilisé pour stocker les données.
Spring Boot	Framework Java pour le développement backend de l'application.
Spring Cloud	Suite d'outils pour le développement d'applications microservices.
Eureka	Service de découverte et registre des microservices (Spring Cloud Netflix Eureka).
Kafka	Apache Kafka, broker de messagerie distribué pour la communication asynchrone entre services.
Thymeleaf	Moteur de template côté serveur pour le rendu des pages HTML.
Chart.js	Bibliothèque JavaScript pour la création de graphiques interactifs.

Abréviation	Description
UI / UX	User Interface / User Experience (Interface Utilisateur / Expérience Utilisateur).
UML	Unified Modeling Language (Langage de Modélisation Unifié), utilisé pour la modélisation du système.
Isolation Forest	Algorithme d'apprentissage automatique non supervisé pour la détection d'anomalies.
One-Class SVM	Support Vector Machine à une classe, algorithme alternatif pour la détection d'anomalies.
MLflow	Plateforme open-source pour la gestion du cycle de vie du machine learning.
scikit-learn	Bibliothèque Python pour l'apprentissage automatique.
Gemini AI	Modèle d'intelligence artificielle de Google utilisé pour la génération de recommandations.
Resilience4j	Bibliothèque Java pour la tolérance aux pannes (circuit breaker, retry, fallback).
BCrypt	Algorithme de hachage de mot de passe utilisé pour la sécurité.
Redis	Système de stockage clé-valeur en mémoire utilisé pour le cache et la gestion de session.
Prometheus	Système de surveillance et d'alerte open-source pour les métriques.
Grafana	Plateforme open-source de visualisation et d'analyse de métriques.
HTTP	HyperText Transfer Protocol (Protocole de Transfert Hyper-Texte).
JSON	JavaScript Object Notation (Format de données structuré).
CSV	Comma-Separated Values (Valeurs séparées par des virgules).
Maven	Outil de gestion et d'automatisation de projets Java.
dotenv-java	Bibliothèque Java pour le chargement de variables d'environnement depuis des fichiers .env.
WebFlux	Framework réactif de Spring pour les applications non-bloquantes.
OpenFeign	Client HTTP déclaratif pour la communication entre microservices (Spring Cloud OpenFeign).

Introduction générale

Dans le contexte actuel de transformation digitale et de prise de conscience croissante de l'importance de la santé et du bien-être personnel, la capacité à analyser et comprendre nos comportements quotidiens représente un enjeu majeur. Les habitudes de vie telles que le sommeil, l'activité physique, l'alimentation, l'hydratation, le temps d'étude et l'humeur influencent significativement notre qualité de vie et notre performance. Cependant, identifier les anomalies dans ces comportements et comprendre leurs causes reste un défi complexe nécessitant des outils avancés d'analyse de données et d'intelligence artificielle.

Pour répondre à ces enjeux, nous avons conçu et réalisé le **Personal Behavior Anomaly Detection System (PBADS)**, un système complet de détection d'anomalies comportementales basé sur une architecture microservices. Cette solution intégrée traite les données quotidiennes des utilisateurs depuis leur saisie jusqu'à la génération d'alertes et de recommandations personnalisées, en passant par l'analyse en temps réel à l'aide d'algorithmes d'apprentissage automatique non supervisés.

L'architecture technique de la solution repose sur une approche moderne et robuste. Le système suit une **architecture microservices** avec 13 services distincts, permettant une séparation claire des responsabilités, une scalabilité élevée et une tolérance aux pannes. Le backend s'appuie sur le framework **Spring Boot** qui fournit des APIs REST complètes, un système de sécurité avancé basé sur JWT pour l'authentification et l'autorisation, ainsi qu'une intégration avec Kafka pour la communication asynchrone entre services. Le frontend utilise **Thymeleaf** avec Spring MVC pour le rendu côté serveur, **Chart.js** pour la visualisation interactive des données, et un système de notifications push en temps réel. Le modèle de données structure de manière cohérente les entités clés du système : **User** pour la gestion des utilisateurs, **DailyData** pour les données quotidiennes, **Alert** pour les alertes générées, et **Recommendation** pour les recommandations générées par l'IA.

La détection d'anomalies constitue le cœur du système, utilisant des algorithmes d'apprentissage automatique non supervisés (**Isolation Forest** et **One-Class SVM**) pour identifier les écarts par rapport aux comportements normaux sans nécessiter de données étiquetées. Le système calcule un score d'anomalie (0-1) et détermine un niveau de sévérité (NORMAL, LOW, MEDIUM, HIGH) pour chaque entrée de données. Lorsqu'une anomalie est détectée, le système génère automatiquement une alerte et sollicite le **service de recommandations** alimenté par **Google Gemini AI** pour produire des suggestions personnalisées visant à améliorer les habitudes de l'utilisateur.

La méthodologie de développement a suivi une approche agile avec une planification structurée en sprints, permettant une adaptation rapide aux besoins évolutifs et une livraison incrémentale des fonctionnalités. Cette approche s'est traduite par une priorisation continue des fonctionnalités, des itérations courtes permettant une validation régulière, et des revues régulières pour ajuster les orientations.

Ce rapport technique documente l'ensemble du projet PBADS, couvrant de manière exhaustive le contexte et les objectifs du projet, l'architecture microservices mise en place,

la modélisation des données et des processus avec les diagrammes UML, l'implémentation détaillée des microservices, les algorithmes de machine learning utilisés, l'intégration avec Google Gemini AI, et les résultats obtenus. Le document met en évidence les apports concrets de la solution : détection automatique des anomalies comportementales, génération de recommandations personnalisées, notifications en temps réel, et visualisation interactive des données via un tableau de bord moderne avec filtres temporels.

Mots-clés : Détection d'anomalies, Machine Learning, Microservices, Spring Boot, Kafka, PostgreSQL, Isolation Forest, One-Class SVM, Google Gemini AI, Recommandations personnalisées, Architecture microservices, Thymeleaf, Chart.js, Resilience4j.

Chapitre 1 : Contexte général et planification du projet

Ce chapitre établit les fondations du projet PBADS en détaillant le contexte général et les problématiques spécifiques liées à la détection d'anomalies comportementales. Nous y exposerons les objectifs clairs du projet, la méthodologie de développement agile choisie, et le plan de travail détaillé. Une analyse des besoins utilisateurs et une stratégie d'architecture microservices seront également abordées pour garantir une exécution robuste et une scalabilité élevée.

1.1 Introduction

Ce chapitre présente le contexte et la problématique du **Personal Behavior Anomaly Detection System (PBADS)**, un système de détection d'anomalies comportementales conçu pour analyser les habitudes quotidiennes des utilisateurs et identifier les écarts par rapport aux comportements normaux. Cette solution technologique a été développée pour répondre au besoin croissant d'outils intelligents permettant de comprendre et d'améliorer nos modes de vie à travers l'analyse de données comportementales et la génération de recommandations personnalisées.

1.2 Contexte général

1.2.1 Environnement et motivation

Dans un monde où la santé et le bien-être personnel deviennent des priorités croissantes, la capacité à suivre et analyser nos comportements quotidiens représente un enjeu majeur. Les habitudes de vie telles que la qualité du sommeil, le niveau d'activité physique, l'alimentation, l'hydratation, le temps consacré à l'étude ou au travail, et l'état émotionnel influencent significativement notre santé physique et mentale, notre productivité et notre qualité de vie globale.

Cependant, identifier les anomalies dans ces comportements et comprendre leurs causes reste un défi complexe. Les méthodes traditionnelles d'auto-évaluation sont souvent subjectives et peu fiables, tandis que l'analyse manuelle de grandes quantités de données comportementales est fastidieuse et sujette à l'erreur humaine. Il existe donc un besoin réel d'outils automatisés capables de détecter les anomalies comportementales de manière objective, de fournir des insights actionnables, et de générer des recommandations personnalisées pour améliorer les habitudes de vie.

1.2.2 Situation initiale et problématique

Avant le développement de PBADS, l'analyse des comportements personnels reposait principalement sur des méthodes manuelles et fragmentées. Les utilisateurs devaient consigner leurs données quotidiennes dans des tableurs Excel ou des applications mobiles isolées, sans capacité d'analyse approfondie ni de détection automatique des anomalies. Cette approche présentait plusieurs limitations critiques :

Limitations observées :

- **Absence de détection automatique** : Les utilisateurs devaient identifier manuellement les anomalies dans leurs données, ce qui est subjectif et peu fiable, surtout pour des patterns complexes ou subtils.
- **Manque d'analyse prédictive** : Aucun système ne permettait d'anticiper les tendances comportementales ou d'identifier les facteurs contribuant aux anomalies.
- **Recommandations génériques** : Les conseils fournis étaient souvent génériques et non adaptés au profil spécifique de chaque utilisateur.
- **Fragmentation des données** : Les données étaient dispersées dans différents outils sans vue d'ensemble cohérente.
- **Absence de notifications proactives** : Les utilisateurs n'étaient pas alertés en temps réel des anomalies détectées.
- **Manque de visualisation interactive** : Les outils existants offraient peu de capacités de visualisation et de filtrage temporel des données.

1.2.3 Objectif de la solution

L'objectif principal est de fournir un **système intégré de détection d'anomalies comportementales** qui automatise l'analyse des données quotidiennes des utilisateurs et génère des insights actionnables. La solution PBADS doit permettre :

- **Saisie structurée des données quotidiennes** : Interface intuitive pour enregistrer les habitudes de vie (sommeil, activité physique, alimentation, hydratation, étude, humeur).
- **Détection automatique des anomalies** : Utilisation d'algorithmes d'apprentissage automatique non supervisés (Isolation Forest, One-Class SVM) pour identifier les écarts par rapport aux comportements normaux sans nécessiter de données étiquetées.
- **Classification des anomalies** : Attribution d'un score d'anomalie (0-1) et d'un niveau de sévérité (NORMAL, LOW, MEDIUM, HIGH) pour chaque détection.
- **Génération d'alertes intelligentes** : Création automatique d'alertes avec identification des facteurs contribuant à l'anomalie.
- **Recommandations personnalisées** : Intégration avec Google Gemini AI pour générer des suggestions adaptées au profil et aux habitudes spécifiques de chaque utilisateur.

- **Visualisation interactive** : Tableau de bord moderne avec graphiques interactifs (Chart.js) et filtres temporels (semaine, mois, année, tout).
- **Notifications en temps réel** : Système de notifications push (toast) pour alerter les utilisateurs immédiatement lorsqu'une nouvelle anomalie est détectée.
- **Gestion des utilisateurs** : Interface d'administration pour la gestion des comptes utilisateurs avec contrôle d'accès basé sur les rôles (ADMIN, USER).

1.2.4 Architecture microservices

Pour répondre à ces objectifs, PBADS suit une **architecture microservices** avec 13 services distincts, permettant une séparation claire des responsabilités, une scalabilité élevée et une tolérance aux pannes. Cette architecture comprend :

- **Couche Frontend** : Service frontend (Thymeleaf) pour l'interface utilisateur
- **Couche API Gateway** : Point d'entrée unique pour toutes les requêtes client
- **Couche Infrastructure** : Eureka (découverte de services), Config Server (configuration centralisée), Kafka (messagerie asynchrone), PostgreSQL (bases de données), Redis (cache), MLflow (gestion des modèles ML)
- **Couche Services Métier** : Auth Service, Data Service, Model Service, Inference Service, Alert Service, Monitoring Service, Recommendation Service

La communication entre services utilise Kafka pour la messagerie asynchrone et HTTP/-REST pour les appels synchrones, avec un mécanisme de fallback pour garantir la résilience en cas de panne de Kafka.

1.3 Conclusion

Ce chapitre a présenté le contexte et les motivations derrière le développement du système PBADS. L'analyse de la situation existante a révélé des lacunes significatives en termes de détection automatique des anomalies, d'analyse prédictive, et de génération de recommandations personnalisées. Ces constats ont motivé le développement d'une solution intégrée basée sur une architecture microservices qui automatise l'analyse des comportements quotidiens et fournit des insights actionnables pour améliorer la qualité de vie des utilisateurs. La méthodologie de développement agile et la planification structurée permettront de répondre efficacement aux enjeux de détection d'anomalies, de personnalisation des recommandations, et de scalabilité du système.

Chapitre 2 : Modélisation

Ce chapitre présente la modélisation complète du système PBADS, incluant l'analyse des besoins utilisateurs, la définition des objectifs, la méthodologie agile adoptée, et la modélisation conceptuelle avec les diagrammes UML (cas d'utilisation, classes, séquences). L'étude des lacunes des solutions existantes a permis d'identifier les besoins critiques en matière de détection automatique d'anomalies, de génération de recommandations personnalisées, et de visualisation interactive des données.

Ce chapitre présente la modélisation complète du système PBADS, incluant l'analyse des besoins, la définition des objectifs, la méthodologie adoptée, et la modélisation conceptuelle avec les diagrammes UML.

2.1 Modélisation du système

Cette section présente les diagrammes de modélisation qui illustrent l'architecture et les interactions du système PBADS.

2.1.1 Contexte général

Situation actuelle et pratique existante

Avant le développement de PBADS, l'analyse des comportements personnels reposait principalement sur des méthodes manuelles et fragmentées. Les utilisateurs devaient consigner leurs données quotidiennes dans des tableurs Excel ou des applications mobiles isolées, sans capacité d'analyse approfondie ni de détection automatique des anomalies.

Lacunes identifiées dans les solutions existantes :

- **Absence de détection automatique** : Les utilisateurs devaient identifier manuellement les anomalies dans leurs données, ce qui est subjectif et peu fiable.
- **Manque d'analyse prédictive** : Aucun système ne permettait d'anticiper les tendances comportementales ou d'identifier les facteurs contribuant aux anomalies.
- **Recommandations génériques** : Les conseils fournis étaient souvent génériques et non adaptés au profil spécifique de chaque utilisateur.
- **Fragmentation des données** : Les données étaient dispersées dans différents outils sans vue d'ensemble cohérente.
- **Absence de notifications proactives** : Les utilisateurs n'étaient pas alertés en temps réel des anomalies détectées.
- **Manque de visualisation interactive** : Les outils existants offraient peu de capacités de visualisation et de filtrage temporel des données.

Objectif de la solution

L'objectif principal est de fournir un système intégré de détection d'anomalies comportementales qui automatise l'analyse des données quotidiennes des utilisateurs et génère des insights actionnables. La solution PBADS doit permettre la saisie structurée des données quotidiennes, la détection automatique des anomalies à l'aide d'algorithmes d'apprentissage automatique non supervisés, la génération d'alertes intelligentes avec identification des facteurs contribuant à l'anomalie, et la production de recommandations personnalisées via l'intégration avec Google Gemini AI.

2.1.2 Enjeux et objectifs

Les enjeux majeurs de PBADS sont : détection automatique des anomalies comportementales, génération de recommandations personnalisées, notifications en temps réel, et visualisation interactive des données. Ces enjeux se traduisent par les objectifs concrets suivants :

- **Gestion des données quotidiennes** : Saisie structurée des habitudes de vie (sommeil, activité physique, alimentation, hydratation, étude, humeur).
- **Détection automatique d'anomalies** : Utilisation d'algorithmes ML (Isolation Forest, One-Class SVM) pour identifier les écarts par rapport aux comportements normaux.
- **Génération d'alertes intelligentes** : Création automatique d'alertes avec niveaux de sévérité (NORMAL, LOW, MEDIUM, HIGH) et facteurs contribuant à l'anomalie.
- **Recommandations personnalisées** : Intégration avec Google Gemini AI pour générer des suggestions adaptées au profil de chaque utilisateur.
- **Visualisation interactive** : Tableau de bord avec graphiques interactifs (Chart.js) et filtres temporels (semaine, mois, année, tout).
- **Notifications en temps réel** : Système de notifications push (toast) pour alerter les utilisateurs immédiatement.
- **Gestion des utilisateurs** : Interface d'administration pour la gestion des comptes avec contrôle d'accès basé sur les rôles (ADMIN, USER).

2.1.3 Méthodologie adoptée

Pour ce projet, nous avons opté pour une méthodologie agile, qui favorise un développement itératif et incrémental. Chaque itération permet de livrer des fonctionnalités prêtes à l'usage tout en recueillant régulièrement des retours des utilisateurs.

Le cycle de vie du projet

Le cycle de vie de notre projet comprend les activités suivantes :

1. **Analyse des besoins et faisabilité** : Identification des besoins fonctionnels et non fonctionnels pour la détection d'anomalies comportementales.
2. **Planification et priorisation** : Création du backlog produit, définition des priorités et planification des itérations.
3. **Conception** : Élaboration des spécifications de l'architecture microservices, des bases de données et des interfaces utilisateur.
4. **Développement** : Implémentation progressive des microservices avec des revues de code et une intégration continue.
5. **Tests** : Vérification de chaque fonctionnalité par des tests unitaires, fonctionnels et d'intégration.
6. **Documentation** : Rédaction des informations nécessaires à l'utilisation, la maintenance et les futures évolutions.
7. **Livraison et déploiement** : Livraison progressive des incréments et mise en production du système.

Cette approche agile assure une meilleure collaboration entre les parties prenantes et une flexibilité accrue face aux changements, tout en maintenant un haut niveau de qualité.

2.2 Étude Conceptuelle

Cette section présente l'étude conceptuelle du système avec les diagrammes UML qui modélisent les interactions et la structure de la solution PBADS.

2.2.1 Méthodologie

L'étude conceptuelle suit une approche structurée basée sur l'analyse des besoins utilisateurs et la modélisation des interactions système. Cette méthodologie permet de capturer les exigences fonctionnelles et non-fonctionnelles, d'identifier les acteurs clés, et de concevoir une architecture cohérente et évolutive.

2.2.2 Les acteurs principaux

Le système PBADS implique plusieurs acteurs aux rôles et responsabilités distincts. Le tableau suivant présente l'ensemble des acteurs identifiés avec leurs fonctionnalités principales.

Acteur	Rôle	Fonctionnalités principales
Utilisateur	Utilisateur standard	Connexion, inscription, saisie des données quotidiennes, consultation du tableau de bord, visualisation des alertes, consultation des recommandations, filtrage des graphiques
Administrateur	Gestionnaire système	Hérite de toutes les fonctionnalités utilisateur + gestion des utilisateurs (CRUD), activation/désactivation des comptes, consultation de tous les utilisateurs
Modèle ML	Système autonome	Entraînement du modèle, chargement du modèle, effectuation d'inférence, détection d'anomalies, stockage et versioning du modèle, évaluation du modèle

Tab. 2.1 : Acteurs principaux du système PBADS

2.2.3 Le diagramme de cas d'utilisation

Le diagramme de cas d'utilisation ci-dessous présente l'ensemble des fonctionnalités du système PBADS, organisées par acteur et illustrant les relations d'héritage entre l'Administrateur et l'Utilisateur.

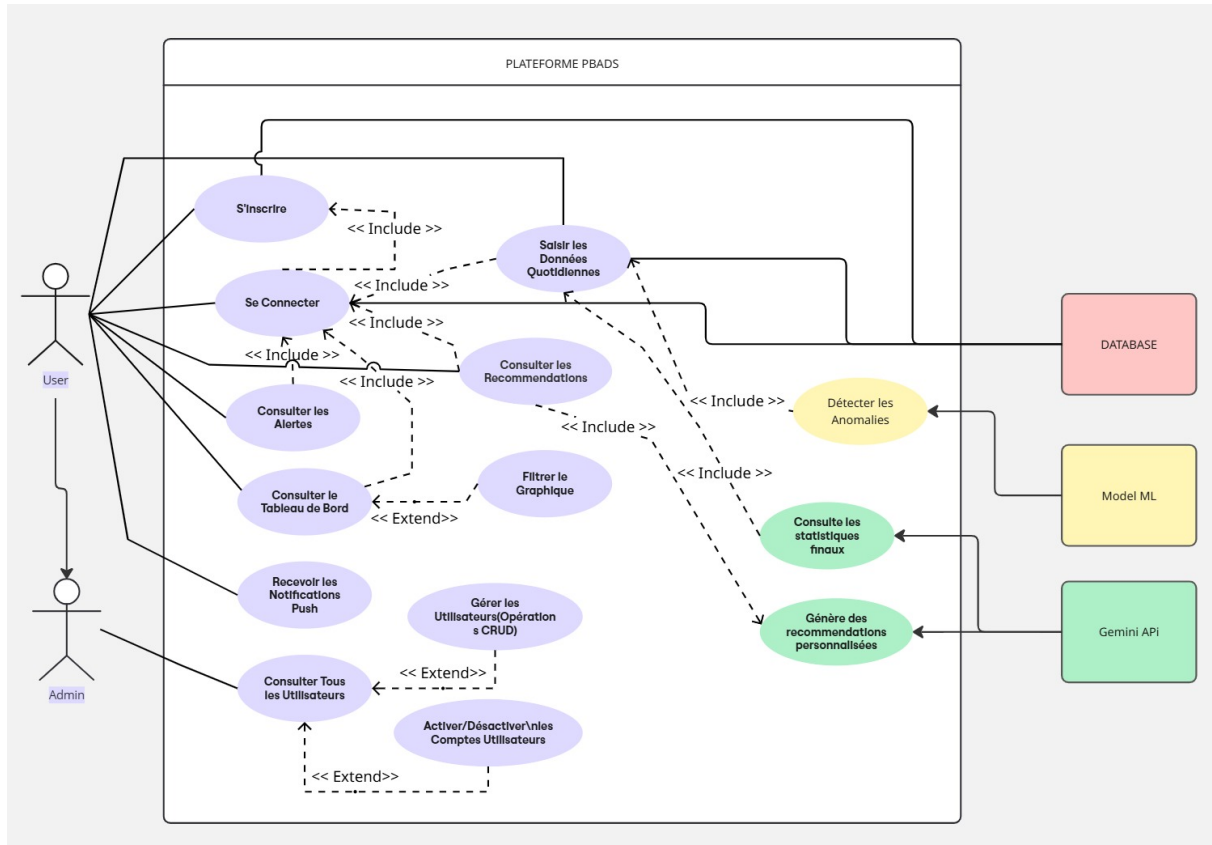


Fig. 2.1 : Diagramme de cas d'utilisation du système PBADS

Description du diagramme de cas d'utilisation :

Le diagramme illustre trois acteurs principaux : **Utilisateur**, **Administrateur** (qui hérite de toutes les fonctionnalités utilisateur), et **Modèle ML**. Les cas d'utilisation sont organisés en trois groupes fonctionnels :

- **Fonctionnalités Utilisateur :** Connexion, inscription, consultation du tableau de bord, saisie des données quotidiennes, consultation des alertes, consultation des recommandations, réception des notifications push, filtrage des graphiques.
- **Fonctionnalités Administrateur :** Gestion des utilisateurs (CRUD), activation/désactivation des comptes, consultation de tous les utilisateurs.
- **Fonctionnalités Modèle ML :** Entraînement, chargement, inférence, détection d'anomalies, stockage, versioning, évaluation.

Les relations entre cas d'utilisation incluent des dépendances <<include>> (par exemple, la saisie de données inclut la détection d'anomalies) et des extensions <<extend>> (par exemple, la consultation du tableau de bord peut étendre le filtrage des graphiques).

2.2.4 Diagramme de classes

Le diagramme de classes ci-dessous présente l'architecture complète du système PBADS, illustrant les entités métier principales et leurs relations. Cette modélisation reflète fidèlement l'implémentation technique réalisée avec Spring Boot et JPA.

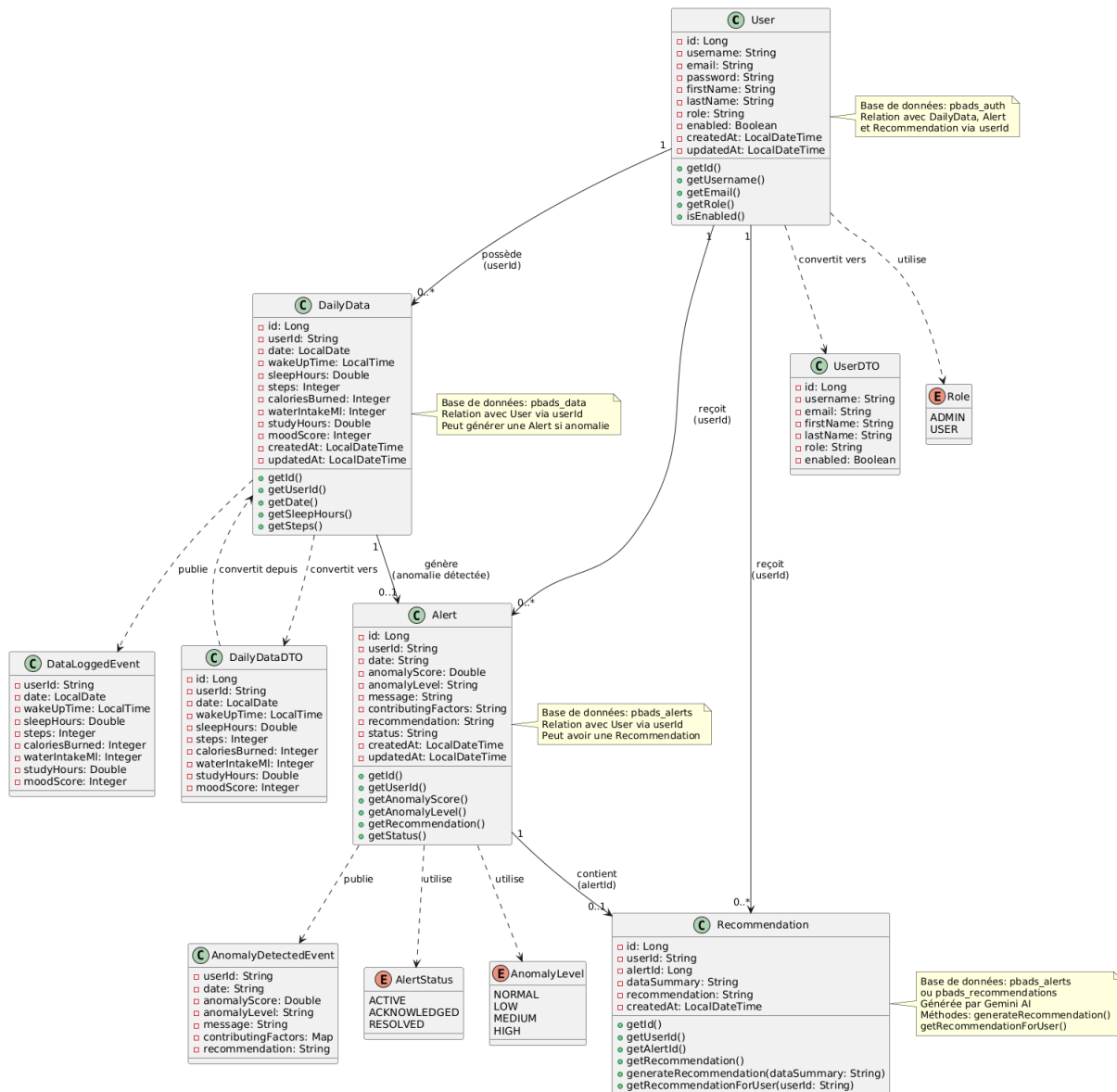


Fig. 2.2 : Diagramme de classes du système PBADS

Description du diagramme de classes :

Le modèle de données s'articule autour de trois entités principales : **User**, **DailyData**,

et **Alert**. Chaque entité est stockée dans une base de données PostgreSQL distincte selon le principe de l'architecture microservices (une base de données par service).

Entités principales :

- **User** : Gestion des comptes utilisateurs avec authentification JWT, incluant les attributs username, email, password (hashé avec BCrypt), role (ADMIN ou USER), enabled (actif/désactivé), et les timestamps createdAt et updatedAt.
- **DailyData** : Données quotidiennes des utilisateurs avec userId, date, wakeUpTime, sleepHours, steps, caloriesBurned, waterIntakeMl, studyHours, moodScore, et les timestamps createdAt et updatedAt.
- **Alert** : Alertes générées lors de la détection d'anomalies avec userId, date, anomalyScore (0-1), anomalyLevel (NORMAL, LOW, MEDIUM, HIGH), message, contributingFactors (JSON), recommendation (générée par Gemini AI), status (ACTIVE, ACKNOWLEDGED, RESOLVED), et les timestamps createdAt et updatedAt.
- **Recommendation** : Entité représentant les recommandations générées par Google Gemini AI, avec userId, alertId, dataSummary, recommendation, et createdAt. Les méthodes incluent generateRecommendation() et getRecommendationForUser().

Relations clés :

Le diagramme illustre les relations cardinales importantes :

- Un **User** (1) peut avoir plusieurs **DailyData** (0..*) via userId
- Un **User** (1) peut recevoir plusieurs **Alert** (0..*) via userId
- Un **User** (1) peut recevoir plusieurs **Recommendation** (0..*) via userId
- Une **DailyData** (1) peut générer une **Alert** (0..1) si une anomalie est détectée
- Une **Alert** (1) peut avoir une **Recommendation** (0..1) associée via alertId

Énumérations :

Le système utilise plusieurs énumérations pour standardiser les valeurs : **Role** (ADMIN, USER), **AlertStatus** (ACTIVE, ACKNOWLEDGED, RESOLVED), et **AnomalyLevel** (NORMAL, LOW, MEDIUM, HIGH).

Cette architecture garantit l'intégrité des données, la cohérence des relations et la flexibilité nécessaire pour supporter l'évolution future du système.

2.2.5 Les diagrammes de séquence

Cette section présente les flux d'interactions dynamiques entre les différents acteurs du système PBADS. Les diagrammes de séquence suivants décomposent chaque processus en étapes claires, montrant les échanges d'informations entre les acteurs humains et les composants système.

Scénario : Administrateur interagissant avec la plateforme

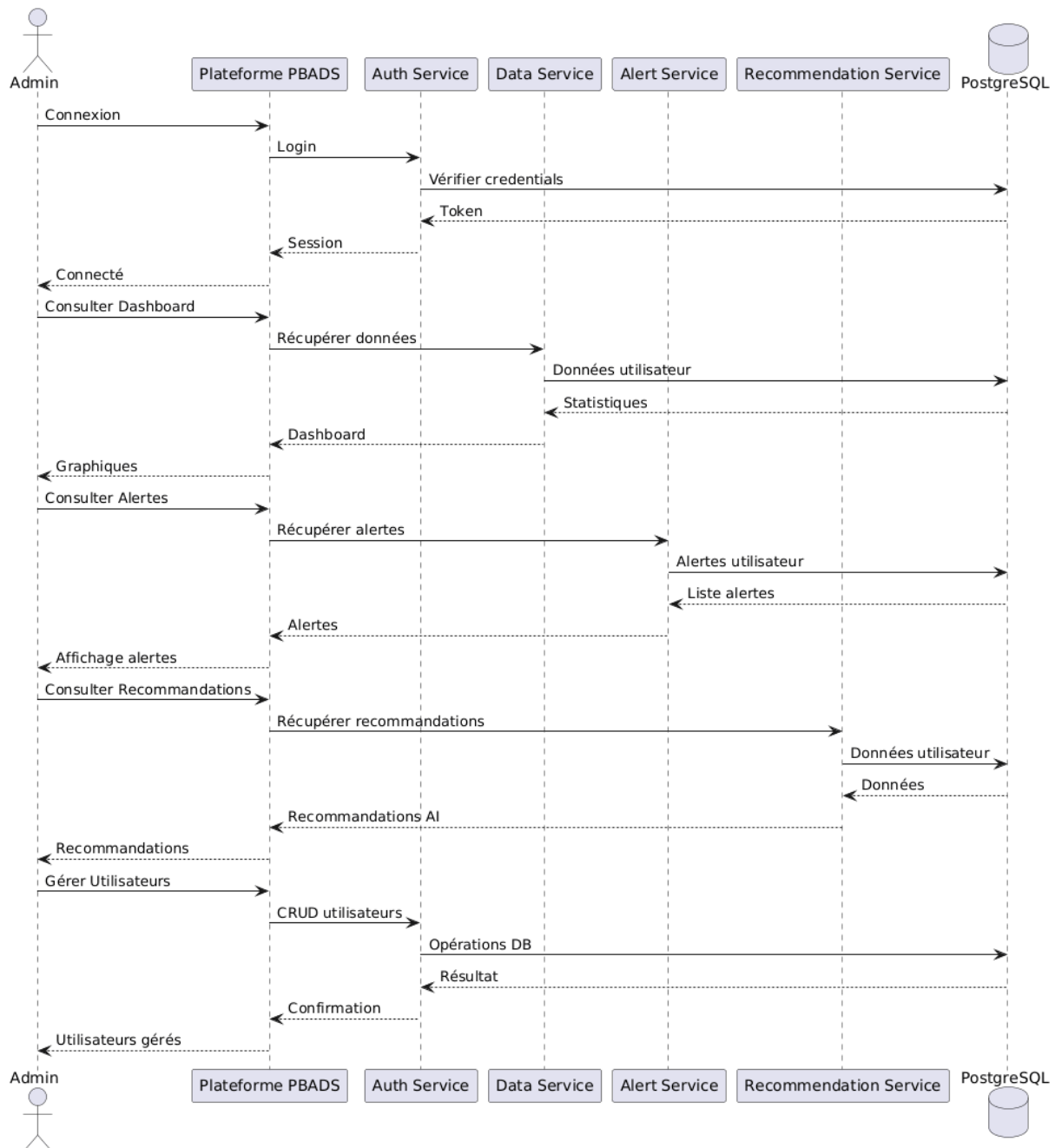


Fig. 2.3 : Diagramme de séquence - Administrateur interagissant avec la plateforme

Scénario Administrateur (Figure 2.3) : Ce diagramme illustre les interactions d'un administrateur avec le système PBADS. Le scénario commence par la **connexion** de l'administrateur via le service d'authentification, qui vérifie les credentials et retourne

un token JWT. L'administrateur peut ensuite **consulter le tableau de bord** en récupérant les données quotidiennes et les recommandations. Il peut également **consulter les alertes** pour visualiser toutes les alertes générées. Enfin, l'administrateur peut **gérer les utilisateurs** en effectuant des opérations CRUD (création, lecture, mise à jour, suppression) et activer/désactiver les comptes utilisateurs.

Scénario : Modèle ML interagissant avec la base de données et la plateforme

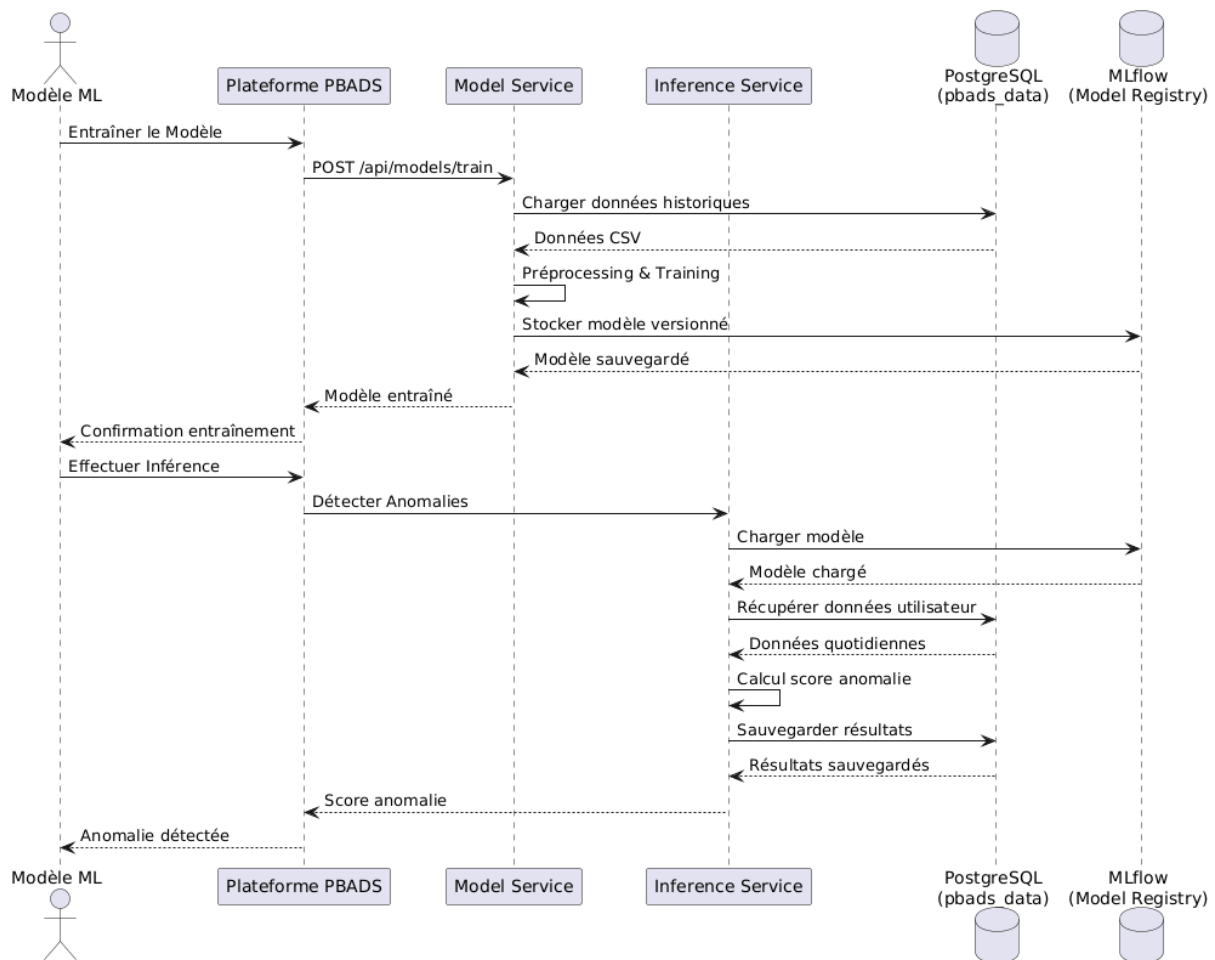


Fig. 2.4 : Diagramme de séquence - Modèle ML interagissant avec la base de données et la plateforme

Scénario Modèle ML (Figure 2.4) : Ce diagramme détaille les interactions du modèle ML avec la plateforme et les bases de données. Le scénario commence par l'**entraînement du modèle** où le Model Service charge les données historiques depuis la base de données, effectue le preprocessing et l'entraînement, puis stocke le modèle versionné dans MLflow.

Lors de l'**inférence**, le Inference Service charge le modèle depuis MLflow, récupère les données utilisateur depuis la base de données, calcule le score d'anomalie, et sauvegarde les résultats. Si une anomalie est détectée, une alerte est générée avec les détails de l'anomalie.

2.3 Conclusion

Ce chapitre a présenté la modélisation complète du système PBADS, couvrant l'analyse des besoins, la définition des objectifs, la méthodologie agile adoptée, et la modélisation conceptuelle détaillée avec les diagrammes UML. L'étude des lacunes des solutions existantes a permis d'identifier les besoins critiques en matière de détection automatique d'anomalies, de génération de recommandations personnalisées, et de visualisation interactive des données.

Les diagrammes UML (cas d'utilisation, classes et séquences) fournissent une vision claire de l'architecture et des interactions du système. Cette modélisation constitue la base solide pour l'implémentation technique qui sera détaillée dans le chapitre suivant.

Chapitre 3 : Interfaces et Réalisation de l'application

Ce chapitre présente les technologies utilisées, l'architecture microservices, et les interfaces développées pour le système PBADS. Il détaille l'implémentation technique des microservices, les choix technologiques (Spring Boot, Kafka, PostgreSQL, scikit-learn, Google Gemini AI), et les interfaces utilisateur avec thème sombre moderne. Cette approche modulaire facilite la maintenance, l'évolution et la scalabilité de la solution.

Ce chapitre présente les technologies utilisées, l'architecture technique, et les interfaces développées pour le système PBADS.

3.1 Technologies et Architecture

3.1.1 Architecture microservices

Notre système suit une architecture microservices moderne, séparant clairement les responsabilités entre les différents services pour garantir la scalabilité, la maintenabilité et la tolérance aux pannes.

Composants de l'architecture :

- **Frontend (Thymeleaf) :** Interface utilisateur développée avec Thymeleaf et Chart.js
- **Backend (Spring Boot) :** APIs REST développées avec Spring Boot
- **Bases de données (PostgreSQL) :** Bases de données relationnelles avec JPA/-Hibernate
- **Sécurité :** Authentification JWT et contrôle d'accès basé sur les rôles
- **Communication :** Kafka pour la messagerie asynchrone et HTTP/REST pour les appels synchrones
- **Machine Learning :** Python (scikit-learn) pour l'entraînement et l'inférence des modèles
- **IA :** Google Gemini AI pour la génération de recommandations personnalisées

3.1.2 Technologies Frontend

Thymeleaf

Thymeleaf constitue le moteur de template côté serveur pour notre interface utilisateur, offrant une intégration native avec Spring MVC.

Composants principaux :

- **Templates :** Pages HTML avec syntaxe Thymeleaf pour le rendu dynamique

- **Fragments** : Composants réutilisables (navbar, footer)
- **Expressions** : Accès aux données du modèle et aux variables de session
- **Sécurité** : Intégration avec Spring Security pour la protection des routes

Chart.js

Chart.js fournit une bibliothèque de graphiques JavaScript pour la visualisation interactive des données comportementales.

Composants utilisés :

- **Graphiques linéaires** : Visualisation des tendances comportementales dans le temps
- **Filtres temporels** : Filtrage par semaine, mois, année, ou toutes les données
- **Animations** : Transitions fluides lors du changement de filtres
- **Thème sombre** : Interface adaptée avec palette de couleurs personnalisée

3.1.3 Technologies Backend

Spring Boot Framework

Spring Boot constitue la base de nos APIs REST, offrant une configuration simplifiée et des fonctionnalités enterprise robustes.

Modules Spring utilisés :

- **Spring Web** : Développement d'API REST avec annotations
- **Spring Security** : Gestion de la sécurité et de l'authentification
- **Spring Data JPA** : Abstraction de la couche de données
- **Spring Cloud Gateway** : Point d'entrée unique pour toutes les requêtes
- **Spring Cloud Eureka** : Découverte de services
- **Spring Cloud Config** : Configuration centralisée
- **Spring Cloud OpenFeign** : Client HTTP déclaratif pour la communication entre services

Kafka

Apache Kafka est utilisé comme broker de messagerie pour la communication asynchrone entre services.

Topics utilisés :

- `data.logged.events` : Événements de données quotidiennes enregistrées
- `anomaly.detected.events` : Événements d'anomalies détectées

PostgreSQL Database

PostgreSQL a été choisi comme base de données relationnelle pour sa robustesse et ses performances.

Bases de données :

- `pbads_auth` : Données d'authentification et utilisateurs
- `pbads_data` : Données quotidiennes des utilisateurs
- `pbads_alerts` : Alertes générées

3.1.4 Machine Learning

scikit-learn

scikit-learn est utilisé pour l'entraînement et l'inférence des modèles de détection d'anomalies.

Algorithmes utilisés :

- **Isolation Forest** : Algorithme principal pour la détection d'anomalies
- **One-Class SVM** : Algorithme alternatif avec noyau RBF

MLflow

MLflow est utilisé pour le suivi des expériences ML et le versioning des modèles.

3.2 Interfaces Utilisateur

Cette section présente les interfaces développées pour les utilisateurs et les administrateurs du système PBADS.

3.2.1 Vue Utilisateur

Page d'inscription

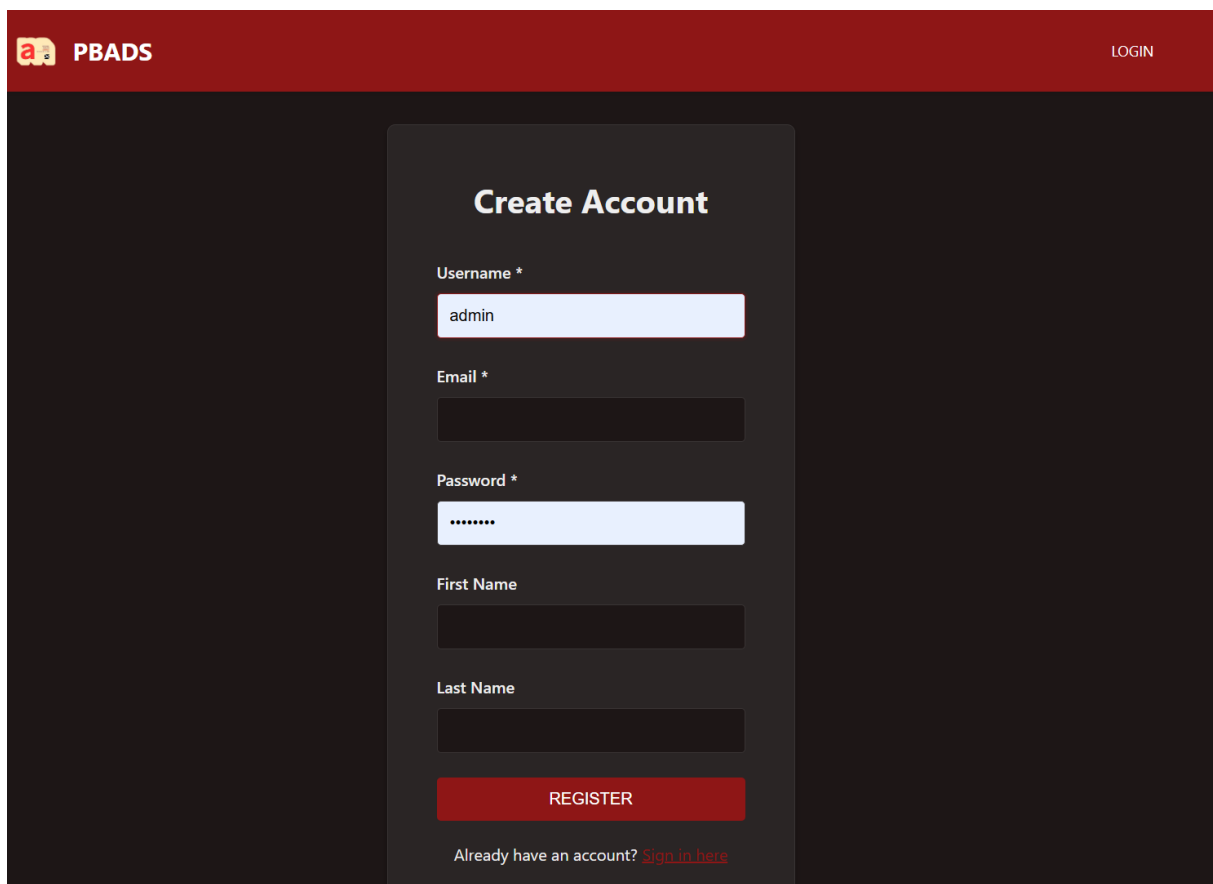
The image shows a web browser window displaying the 'Create Account' page of the PBADS system. The page has a dark theme with a red header bar. In the top left of the header is the PBADS logo, and in the top right is a 'LOGIN' link. The main content area is dark gray and contains a white-bordered box with the title 'Create Account'. Inside this box, there are five input fields: 'Username *' (containing 'admin'), 'Email *' (empty), 'Password *' (masked with dots), 'First Name' (empty), and 'Last Name' (empty). Below these fields is a red 'REGISTER' button. At the bottom of the box, there is a link that says 'Already have an account? [Sign in here](#)'.

Fig. 3.1 : Page d'inscription - Création de compte utilisateur

Cette interface permet aux nouveaux utilisateurs de créer un compte dans le système PBADS. Le formulaire collecte les informations essentielles : nom d'utilisateur, email, mot de passe, et prénom/nom. La page utilise un thème sombre moderne avec une navigation simplifiée (seul le lien "LOGIN" est visible pour les utilisateurs non authentifiés).

Page de connexion

Register here'."/>

Sign In

Username and password are required.

Username *

admin

Password *

SIGN IN

Don't have an account? [Register here](#)

Fig. 3.2 : Page de connexion - Authentification utilisateur

Cette interface permet aux utilisateurs de se connecter au système avec leurs identifiants (nom d'utilisateur et mot de passe). La page suit le même thème sombre cohérent avec le reste de l'application et offre un accès rapide à la page d'inscription pour les nouveaux utilisateurs.

Tableau de bord - Graphiques



Fig. 3.3 : Tableau de bord - Visualisation des tendances comportementales

Cette interface constitue le cœur du système PBADS, présentant une visualisation interactive des données comportementales de l'utilisateur. Le tableau de bord inclut :

- **Graphique des tendances comportementales :** Visualisation des données quotidiennes avec Chart.js
- **Filtres temporels :** Boutons pour filtrer par semaine, mois, année, ou toutes les données
- **Statistiques résumées :** Vue d'ensemble des métriques clés (sommeil moyen, steps moyens, etc.)
- **Thème sombre :** Interface moderne avec palette de couleurs personnalisée (#1a1a1a, #2d2d2d, #8E1616)

Tableau de bord - Alertes et Recommandations

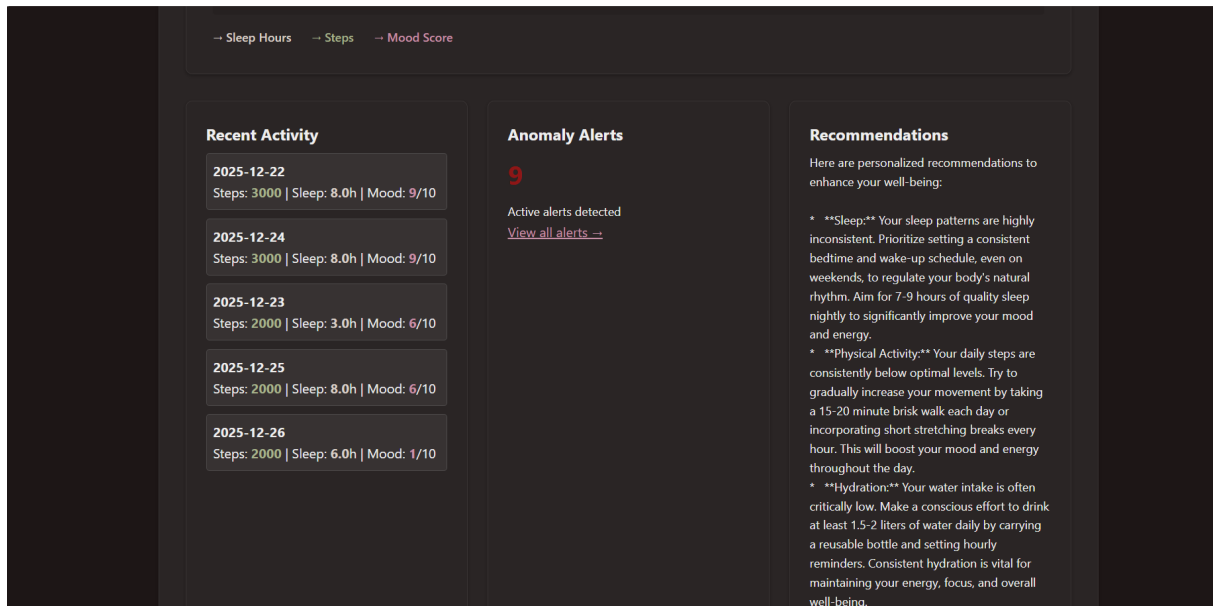


Fig. 3.4 : Tableau de bord - Alertes et recommandations IA

Cette section du tableau de bord présente les alertes générées et les recommandations personnalisées. L'interface affiche :

- **Alertes récentes :** Liste des alertes avec niveaux de sévérité (HIGH, MEDIUM, LOW)
- **Recommandations IA :** Suggestions personnalisées générées par Google Gemini AI
- **Badges de statut :** Indicateurs visuels pour les niveaux d'alerte
- **Design cohérent :** Intégration harmonieuse avec le thème sombre du tableau de bord

Saisie de données quotidiennes

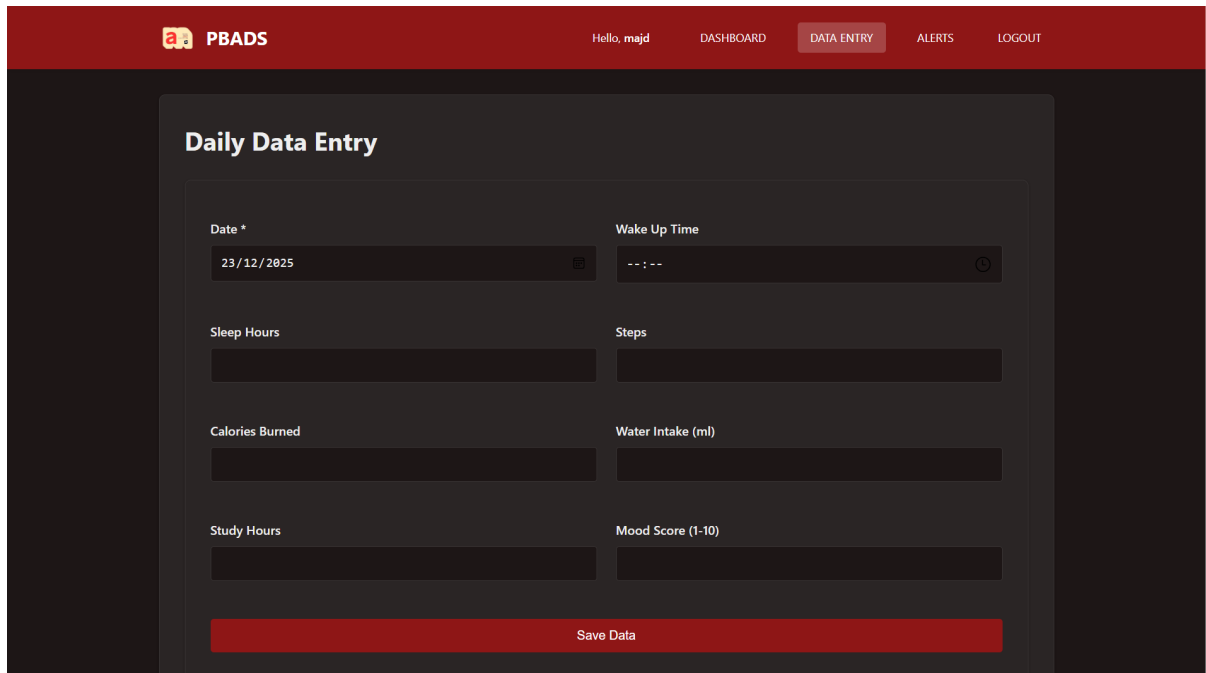
The image shows a web application interface for 'PBADS'. At the top, there is a dark red header bar containing the PBADS logo, the user name 'Hello, majd', and navigation links for 'DASHBOARD', 'DATA ENTRY' (which is highlighted), 'ALERTS', and 'LOGOUT'. Below the header, the main content area is dark gray and features a 'Daily Data Entry' form. The form is organized into two columns. The left column contains fields for 'Date *' (with a calendar icon and the value '23/12/2025'), 'Sleep Hours', 'Calories Burned', and 'Study Hours'. The right column contains fields for 'Wake Up Time' (with a clock icon and the value '--:--'), 'Steps', 'Water Intake (ml)', and 'Mood Score (1-10)'. At the bottom of the form is a prominent red button labeled 'Save Data'.

Fig. 3.5 : Formulaire de saisie des données quotidiennes

Cette interface permet aux utilisateurs de saisir leurs données quotidiennes de manière structurée. Le formulaire inclut :

- **Date :** Sélection de la date pour laquelle les données sont enregistrées
- **Heure de réveil :** Champ pour l'heure de réveil
- **Heures de sommeil :** Nombre d'heures de sommeil
- **Steps :** Nombre de pas effectués dans la journée
- **Calories brûlées :** Nombre de calories brûlées
- **Consommation d'eau :** Quantité d'eau consommée en millilitres
- **Heures d'étude :** Temps consacré à l'étude
- **Score d'humeur :** Évaluation de l'humeur sur une échelle de 1 à 10

Le formulaire utilise une validation côté client et serveur pour garantir l'intégrité des données.

Notification push d'alerte

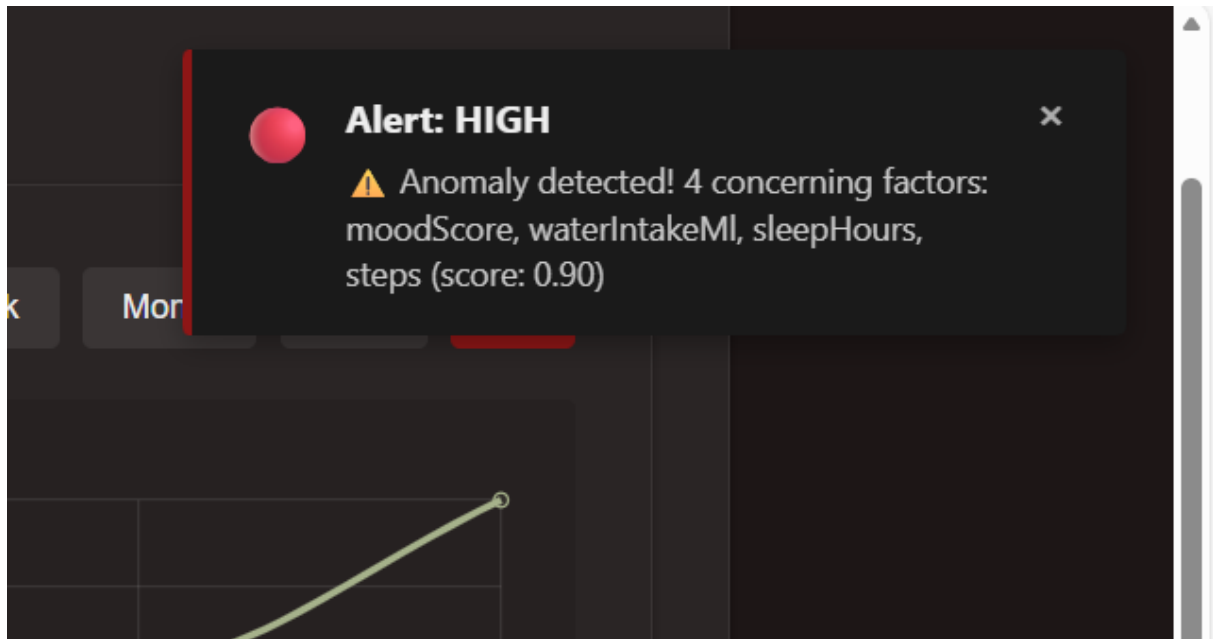


Fig. 3.6 : Notification push - Alerte d'anomalie détectée

Cette interface illustre le système de notifications push (toast) qui alerte les utilisateurs en temps réel lorsqu'une nouvelle anomalie est détectée. Les caractéristiques incluent :

- **Notification éphémère** : Affichage temporaire en haut à droite de l'écran
- **Niveaux de sévérité** : Couleurs différentes selon le niveau (HIGH, MEDIUM, LOW)
- **Message informatif** : Description de l'anomalie détectée
- **Bouton de fermeture** : Possibilité de fermer manuellement la notification
- **Animation** : Transitions fluides pour l'apparition et la disparition

Le système poll le service d'alertes toutes les 5 secondes pour détecter les nouvelles alertes et afficher les notifications automatiquement.

Page des alertes

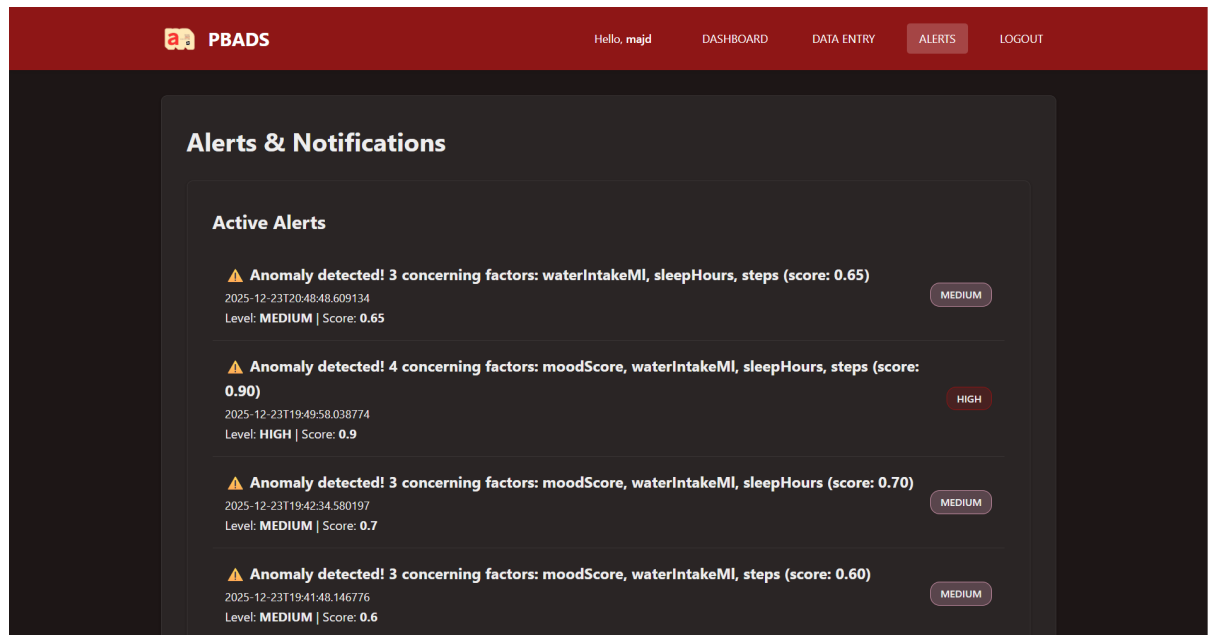


Fig. 3.7 : Page de consultation des alertes

Cette interface permet aux utilisateurs de consulter toutes leurs alertes de manière organisée. La page affiche :

- **Liste complète des alertes** : Toutes les alertes générées pour l'utilisateur
- **Informations détaillées** : Date, score d'anomalie, niveau, message
- **Statut des alertes** : Indicateurs visuels pour ACTIVE, ACKNOWLEDGED, RESOLVED
- **Organisation temporelle** : Alertes triées par date (plus récentes en premier)
- **Design cohérent** : Thème sombre avec badges colorés pour les niveaux de sévérité

3.2.2 Vue Administrateur

Gestion des utilisateurs

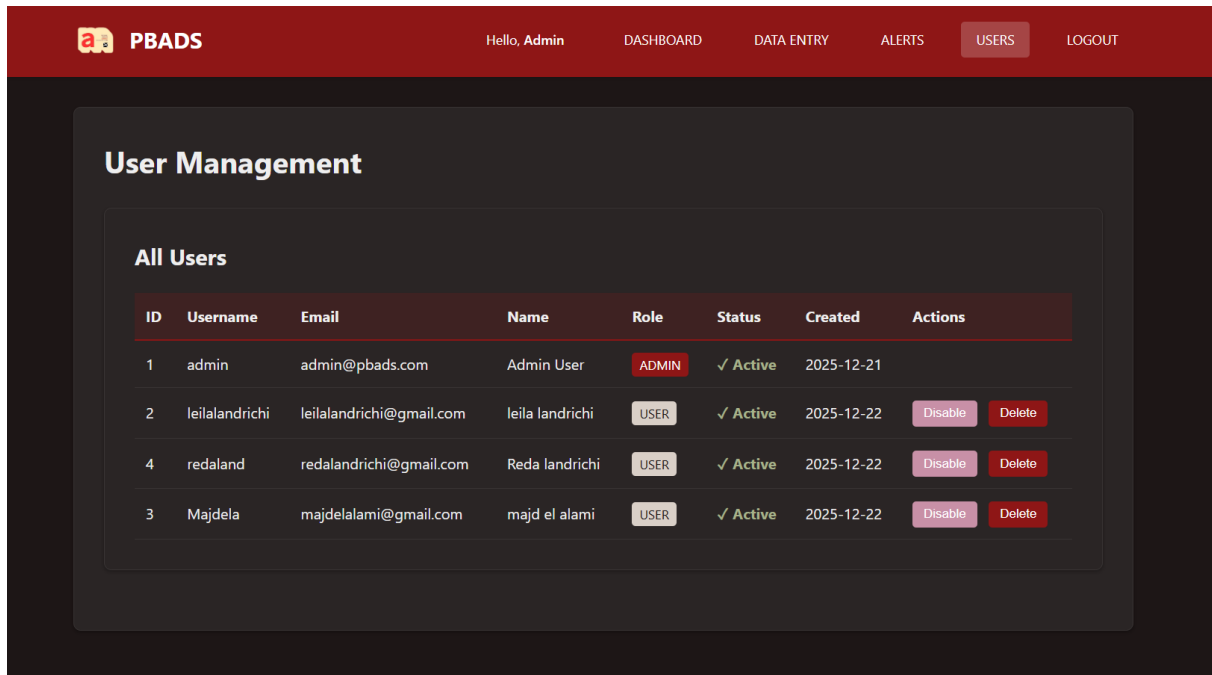


Fig. 3.8 : Interface de gestion des utilisateurs (Administrateur)

Cette interface permet à l'administrateur de gérer tous les utilisateurs du système. Les fonctionnalités incluent :

- **Liste complète des utilisateurs** : Affichage de tous les comptes utilisateurs
- **Opérations CRUD** : Création, lecture, mise à jour et suppression d'utilisateurs
- **Gestion des rôles** : Attribution des rôles ADMIN ou USER
- **Activation/Désactivation** : Contrôle de l'état actif/inactif des comptes
- **Informations détaillées** : Affichage des données utilisateur (nom d'utilisateur, email, rôle, statut)
- **Interface moderne** : Tableau responsive avec actions rapides

L'administrateur hérite de toutes les fonctionnalités utilisateur (tableau de bord, saisie de données, consultation des alertes) en plus de ces fonctionnalités de gestion.

3.3 Conclusion

Ce chapitre a présenté l'implémentation technique complète du système PBADS, couvrant l'architecture microservices, les technologies utilisées, et les interfaces développées pour les utilisateurs et les administrateurs.

L'architecture technique choisie, basée sur Spring Boot et Thymeleaf, offre une base solide et moderne pour le développement d'APIs REST robustes et performantes. L'utilisation de PostgreSQL avec JPA/Hibernate garantit une gestion efficace des données, tandis que l'intégration Kafka assure la communication asynchrone entre services. L'authentification JWT et le contrôle d'accès basé sur les rôles assurent une sécurité renforcée.

Les interfaces développées privilégient l'expérience utilisateur avec un thème sombre moderne, des graphiques interactifs avec Chart.js, et un système de notifications push en temps réel. Chaque rôle dispose d'interfaces adaptées à ses besoins spécifiques, facilitant ainsi l'adoption et l'efficacité du système.

Cette approche modulaire facilite la maintenance, l'évolution et l'adaptation de la solution aux besoins changeants, tout en garantissant une implémentation cohérente et robuste du système PBADS.

Conclusion générale

La réalisation du **Personal Behavior Anomaly Detection System (PBADS)** a constitué une étape majeure de notre parcours académique.

Ce projet nous a confrontés à un défi essentiel : concevoir une solution numérique capable d'analyser les comportements quotidiens des utilisateurs, de détecter automatiquement les anomalies à l'aide d'algorithmes d'apprentissage automatique, et de générer des recommandations personnalisées pour améliorer la qualité de vie.

Nous avons conçu, développé et mis en service un système fonctionnel basé sur une architecture microservices qui automatise la détection d'anomalies comportementales, génère des alertes intelligentes avec niveaux de sévérité, et fournit des recommandations personnalisées via l'intégration avec Google Gemini AI. L'application permet désormais une analyse en temps réel des habitudes de vie, un suivi précis des tendances comportementales, ainsi qu'une visualisation interactive des données via un tableau de bord moderne avec filtres temporels.

Tout au long de ce projet, nous avons su relever plusieurs défis techniques et organisationnels, notamment la modélisation des besoins utilisateurs avec les diagrammes UML, la conception d'une architecture microservices robuste avec Spring Boot, l'intégration de mécanismes d'authentification JWT et de contrôle d'accès basé sur les rôles, l'implémentation de la communication asynchrone avec Kafka et synchrone avec HTTP/REST, l'intégration d'algorithmes de machine learning (Isolation Forest, One-Class SVM) pour la détection d'anomalies, l'intégration avec Google Gemini AI pour la génération de recommandations, et l'optimisation de l'ergonomie des interfaces avec un thème sombre moderne et des graphiques interactifs.

Le respect des contraintes de délai, la rigueur dans la planification agile par itérations, et l'esprit collaboratif au sein de notre équipe ont été des facteurs clés dans l'atteinte de nos objectifs.

Cette expérience nous a permis de renforcer nos compétences en développement fullstack, en architecture microservices, en machine learning, ainsi qu'en intégration d'APIs d'intelligence artificielle.

Elle nous a aussi sensibilisés à l'importance d'une documentation claire, de cycles de test rigoureux, de la modélisation UML pour la compréhension des besoins, et de la prise en compte continue des retours utilisateurs pour garantir la qualité et l'évolutivité du produit.

À l'avenir, nous prévoyons d'enrichir le système avec des fonctionnalités supplémentaires telles qu'une analyse prédictive plus avancée, une intégration avec des dispositifs IoT (montres intelligentes, capteurs), une personnalisation accrue des recommandations basée sur l'historique utilisateur, ou encore une application mobile native pour un suivi encore plus pratique.

En résumé, cette réalisation a été une aventure formatrice et stimulante.

Nous sommes fiers du système que nous avons mis en place et restons motivés à poursuivre son amélioration pour répondre toujours mieux aux besoins d'une détection d'anomalies comportementales moderne, intelligente et accessible.

Bibliographie

- [1] Martin FOWLER et James LEWIS. *Microservices : a definition of this new architectural term*. ThoughtWorks. 2014. URL : <https://martinfowler.com/articles/microservices.html> (visité le 02/09/2025).
- [2] PIVOTAL. *Spring Boot Reference Documentation*. Pivotal Software. 2023. URL : <https://docs.spring.io/spring-boot> (visité le 02/09/2025).
- [3] SPRING. *Spring Cloud Documentation*. VMware. 2023. URL : <https://spring.io/projects/spring-cloud> (visité le 02/09/2025).
- [4] POSTGRESQL GLOBAL DEVELOPMENT GROUP. *PostgreSQL Documentation*. PostgreSQL. 2023. URL : <https://www.postgresql.org/docs> (visité le 02/09/2025).
- [5] APACHE SOFTWARE FOUNDATION. *Apache Kafka Documentation*. Apache Software Foundation. 2023. URL : <https://kafka.apache.org/documentation> (visité le 02/09/2025).
- [6] THYMELEAF. *Thymeleaf Documentation*. Thymeleaf. 2023. URL : <https://www.thymeleaf.org/documentation.html> (visité le 02/09/2025).
- [7] CHART.JS. *Chart.js Documentation*. Chart.js. 2023. URL : <https://www.chartjs.org/docs> (visité le 02/09/2025).
- [8] Ivar JACOBSON, Grady BOOCH et James RUMBAUGH. *The Unified Modeling Language User Guide*. Addison-Wesley, 1999.
- [9] Len BASS, Paul CLEMENTS et Rick KAZMAN. *Software Architecture in Practice*. 3^e éd. Addison-Wesley, 2012.
- [10] Roger S. PRESSMAN et Bruce MAXIM. *Software Engineering : A Practitioner's Approach*. 8^e éd. McGraw-Hill, 2014.
- [11] Ian SOMMERVILLE. *Software Engineering*. 10^e éd. Pearson, 2016.
- [12] Fei Tony LIU, Kai Ming TING et Zhi-Hua ZHOU. *Isolation Forest*. IEEE, 2008, p. 413-422.
- [13] Bernhard SCHÖLKOPF et al. *Estimating the Support of a High-Dimensional Distribution*. T. 13. 7. MIT Press, 2001, p. 1443-1471.
- [14] SCIKIT-LEARN DEVELOPERS. *scikit-learn : Machine Learning in Python*. scikit-learn. 2023. URL : <https://scikit-learn.org/stable> (visité le 02/09/2025).

-
- [15] MLFLOW. *MLflow Documentation*. Databricks. 2023. URL : <https://mlflow.org/docs/latest> (visité le 02/09/2025).
 - [16] GOOGLE. *Google Gemini API Documentation*. Google AI. 2023. URL : <https://ai.google.dev/docs> (visité le 02/09/2025).
 - [17] RESILIENCE4J. *Resilience4j Documentation*. Resilience4j. 2023. URL : <https://resilience4j.readme.io> (visité le 02/09/2025).
 - [18] REDIS. *Redis Documentation*. Redis. 2023. URL : <https://redis.io/docs> (visité le 02/09/2025).
 - [19] PROMETHEUS. *Prometheus Documentation*. Prometheus. 2023. URL : <https://prometheus.io/docs> (visité le 02/09/2025).
 - [20] GRAFANA. *Grafana Documentation*. Grafana Labs. 2023. URL : <https://grafana.com/docs> (visité le 02/09/2025).
 - [21] Varun CHANDOLA, Arindam BANERJEE et Vipin KUMAR. *Anomaly Detection : A Survey*. T. 41. 3. ACM, 2009, 15 :1-15 :58.
 - [22] Victoria HODGE et Jim AUSTIN. *A Survey of Outlier Detection Methodologies*. T. 22. 2. Springer, 2004, p. 85-126.
 - [23] Charu C. AGGARWAL. *Outlier Analysis*. Springer, 2013.
 - [24] IETF. *JSON Web Token (JWT)*. IETF. 2023. URL : <https://jwt.io> (visité le 02/09/2025).
 - [25] BCrypt. *bcrypt Algorithm*. OpenBSD. 2023. URL : <https://en.wikipedia.org/wiki/Bcrypt> (visité le 02/09/2025).
 - [26] NETFLIX. *Netflix Eureka*. Netflix. 2023. URL : <https://github.com/Netflix/eureka> (visité le 02/09/2025).
 - [27] SPRING. *Spring Cloud OpenFeign*. VMware. 2023. URL : <https://spring.io/projects/spring-cloud-openfeign> (visité le 02/09/2025).
 - [28] SPRING. *Spring WebFlux*. VMware. 2023. URL : <https://docs.spring.io/spring-framework/reference/web/webflux.html> (visité le 02/09/2025).
 - [29] DOTENV-JAVA. *dotenv-java Documentation*. dotenv-java. 2023. URL : <https://github.com/cdimascio/dotenv-java> (visité le 02/09/2025).
 - [30] Kent BECK et al. *Manifesto for Agile Software Development*. Agile Alliance, 2001.
 - [31] Chris RICHARDSON. *Microservices Patterns : With Examples in Java*. Manning Publications, 2018.
 - [32] Sam NEWMAN. *Building Microservices : Designing Fine-Grained Systems*. 2^e éd. O'Reilly Media, 2021.

Résumé

Ce projet a été réalisé dans le cadre du développement d'un système de détection d'anomalies comportementales personnelles (PBADS). Il vise à analyser les habitudes quotidiennes des utilisateurs et à détecter automatiquement les anomalies à l'aide d'algorithmes d'apprentissage automatique non supervisés. Basé sur une architecture microservices avec 13 services distincts, le système combine performance, évolutivité et sécurité. Sa réalisation avec Spring Boot (backend), Thymeleaf (frontend), Kafka (messagerie), PostgreSQL (bases de données), scikit-learn (ML), et Google Gemini AI (recommandations) permet d'offrir une solution intuitive, fiable et adaptée aux besoins de suivi comportemental personnel.

Abstract

This project was carried out within the framework of developing a Personal Behavior Anomaly Detection System (PBADS). Its objective is to analyze users' daily habits and automatically detect anomalies using unsupervised machine learning algorithms. Built on a microservices architecture with 13 distinct services, the system ensures performance, scalability, and security. Developed using Spring Boot (backend), Thymeleaf (frontend), Kafka (messaging), PostgreSQL (databases), scikit-learn (ML), and Google Gemini AI (recommendations), it provides an intuitive, reliable, and efficient solution tailored to the needs of personal behavioral tracking.
